LSTM Based Regression for Stock Price Prediction

Aditya Rao Ghodke Arvinder Singh Mundra Dharma Pokhrel

1 Project Overview & Problem Statement

Financial markets are among the most complex and dynamic systems studied in quantitative disciplines. The core motivation for this project is to develop a deep-learning based predictive model that can capture temporal dependencies in financial time series which traditional statistical techniques fail to understand.

This project focuses on predicting the daily closing price of Apple Inc. (AAPL) stock using a Long Short-Term Memory (LSTM) neural network which is a specialized class of Recurrent Neural Networks (RNNs) designed to learn long-range sequential patterns.

The refined problem statement is:

"Given a window of the last 30 trading days of historical closing prices, predict the next day's closing price for AAPL with minimum mean absolute error."

Compared to the initial concept, the final scope sharpened the input representation from raw price sequences to a MinMax-scaled windowed approach, added an Enhanced LSTM architecture comparison, and used a rigorous data-leakage-free normalization pipeline. From the progress report stage, we further refined the Enhanced LSTM architecture by increasing hidden layers from 3 to 5 and adding dropout regularization based on mid-project validation results. The real-world significance is clear: retail and institutional investors benefit from better short-term price forecasts for risk management, portfolio re balancing, and algorithmic trading strategies. Even a small improvement in predictive accuracy over a moving-average baseline can translate to meaningful financial gains at scale.

2 Data Acquisition, Sources & Description

2.1 Data Source

All data was collected using the yfinance Python library, which provides a programmatic interface to Yahoo Finance's historical market data. No SQL or NoSQL database was required for this project; the yfinance API returns data directly as pandas DataFrame. The dataset was saved locally as CSV file for reproducibility, and no JSON or XML parsing was necessary. The download was parameterized as follows:

```
Ticker symbol: AAPL (Apple Inc.)
Start date: January 1, 2010
End date: February 25, 2026
API call: yf.download(ticker, start=start_date, end=end_date, progress=False)
```

2.2 Dataset Characteristics and Summary Statistics

After downloading, only two columns were retained: Date (the trading day index) and Close (the adjusted closing price in USD). The dataset shape, types, and summary statistics are:

Table 1: Dataset summary statistics for the AAPL closing price series

Attribute	Value
Total Rows	4,060 trading days
Date Range	2010-01-04 – 2026-02-24
Feature Retained	Close price (USD)
Feature Type	Continuous numeric (float64)
Index Type	DatetimeIndex
Missing Values	0 (confirmed via isnull().sum())

Table 2: Descriptive Statistics – AAPL Daily Close Price (USD)

Count	4,060	Mean	\$79.08	Std	\$76.62
Min	\$5.75	25%	\$18.35	Median	\$39.54
75%	\$144.31	Max	\$285.92	Skewness	+1.3

2.3 Limitations

(i) Univariate: no macro, volume, or sentiment signals.
(ii) Single asset: findings may not generalise across sectors.
(iii) Adjusted prices may embed discontinuities from 2020 stock split.

3 Data Preprocessing & Cleaning

3.1 Missing Value Audit

A missing value audit was performed immediately after download. The result confirmed zero missing values, which is expected because Yahoo Finance already filters non-trading days (weekends and U.S. market holidays) from the returned series. No imputation was required.

```
print(df.isnull().sum())
```

3.2 Outlier Detection

A horizontal boxplot was generated as shown in Figure 1 for the Close price column to visually detect outliers. Because AAPL's price changed dramatically over 16 years from single digits to over $250, the boxplot reveals a heavily right-skewed distribution with many upper-end statistical 'outliers'.
However, these represent genuine market behavior (rapid appreciation post-COVID stimulus and iPhone product cycles) rather than data errors. No data points were removed; instead, MinMax normalization was used to compress the range without distorting temporal ordering.

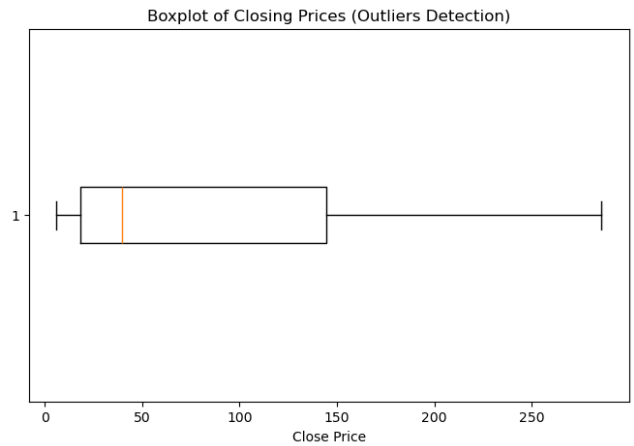


Figure 1: Boxplot of AAPL Daily Closing Prices

3.3 Feature Engineering: Log Returns

Daily log returns were computed for stationarity analysis and EDA:

```
df["log_return"] = np.log(df["price"] / df["price"].shift(1))
df.dropna(inplace=True)
```

This feature is not fed into the model directly but provide essential insights into the volatility structure and momentum of the series during EDA.

3.4 Normalization & Data Leakage Prevention

MinMaxScaler from scikit-learn was applied separately to X and y to scale all values to [0, 1]. A critical design decision was made to fit the scalers only on the training portion of the dataset and then transform validation and test sets using those fitted parameters. This prevents data leakage; if the scaler had been fit on the full dataset, future price information would have implicitly leaked into the training process, producing artificially optimistic metrics.

In the initial prototype, this mistake was made and identified when validation loss appeared suspiciously low. The fix was to refit the scalers exclusively on training data before any split-dependent transformation.

MinMax scaling was *fit only on the training split* and *applied* (not re-fit) to val/test to prevent look-ahead bias:

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 1))
train_sc = scaler.fit_transform(train_prices)
val_sc = scaler.transform(val_prices)
test_sc = scaler.transform(test_prices)
```

3.5 Sliding-Window Sequence Construction

The LSTM model requires fixed-length sequences as input. The first n=30 time steps form the input sequence X, and the 31st value is the prediction target y. This yields shape (N, 30) for X and (N,) for y, where N = 4,029 samples.

```
def df_to_windowed_np(df, n=30):
    close_values = df['Close'].to_numpy().flatten()
    X = np.lib.stride_tricks.sliding_window_view(close_values, window_shape=n+1)
    X_values = X[:, :-1]
    Y_values = X[:, -1]
    dates = df.index[n:]
    return dates, X_values, Y_values
```

3.6 Chronological Train / Validation / Test Split

No shuffling is applied; splits preserve temporal order. Figure 2 visualises the three segments.

Metric	Train	Validation	Test
Proportion	80%	10%	10%
Approx. Rows	3224	403	403

Table 3: Data Split Summary

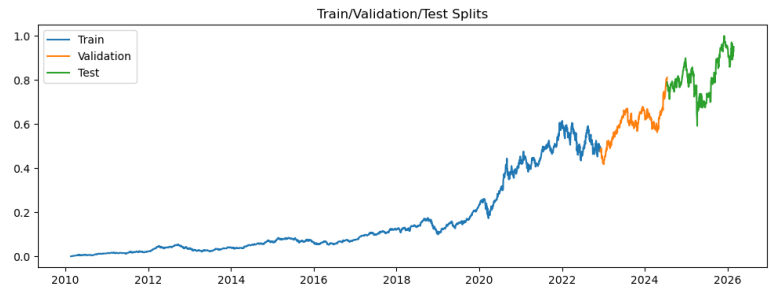


Figure 2: Chronological Train/Val/Test split

After windowing, X was reshaped to (N, 30, 1) to match PyTorch LSTM's expected input format of (batch, sequence_length, input_features). Since the dataset is entirely numeric (continuous float64 closing prices), no categorical encoding or one-hot encoding was required. The reshape from (N, 30) to (N, 30, 1) adds the input-feature dimension required by PyTorch's nn.LSTM, which expects inputs of shape (batch, sequence_length, input_features).

4 Exploratory Data Analysis & Visualization

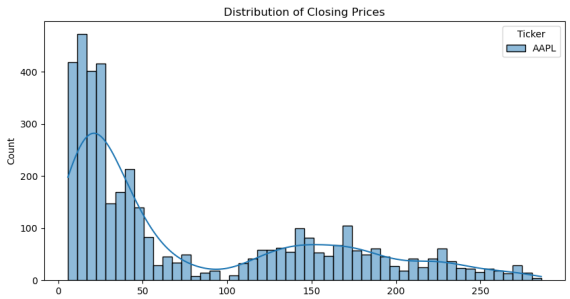
4.1 Temporal Price Trend

Figure 3 plots the full 16-year closing price series. Notable regimes:
(i) 2010–2013 rapid iPhone-era appreciation;
(ii) 2015–2016 short correction;
(iii) 2020 Q1 COVID shock (−30% in 5 weeks);
(iv) 2020–2021 V-shaped recovery;
(v) 2022 macro-driven correction;
(vi) 2023–2026 recovery.

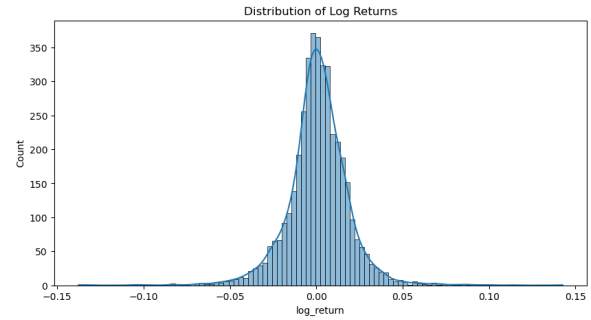
Figure 3: AAPL Daily Closing Price (2010-2026)

4.2 Price Distribution and Log-Return Distribution

Figure 4 (left) shows closing prices are strongly right-skewed (skewness $\approx +1.3$): the bulk of the 4,060 observations lie below \$50, reflecting the pre-appreciation era. Figure 4 (right) shows log returns $r_t = \ln(P_t/P_{t-1})$ are approximately symmetric and near-Gaussian with mild fat tails which is a standard stylised fact of equity markets.



(a) Raw price distribution motivating MinMax normalisation.



(b) Log-return distribution confirms near-stationarity of returns.

Figure 4: Price Distribution and Log-Return Distribution

4.3 Moving Averages (Technical Analysis)

Figure 5 overlays 50-day and 200-day simple moving averages on the raw price. The classic *golden cross* (50-MA crossing above 200-MA) is visible in 2020 and 2023, and the *death cross* in 2022. The widening gap during 2020–2021 reflects the post-COVID bull run.

4.4 Autocorrelation Analysis

Figure 6 shows the ACF of the raw Close price series remaining near 1.0 across all 60 lags, confirming near-unit-root behavior and validating the choice of a 30-day look-back window for the LSTM input sequences.

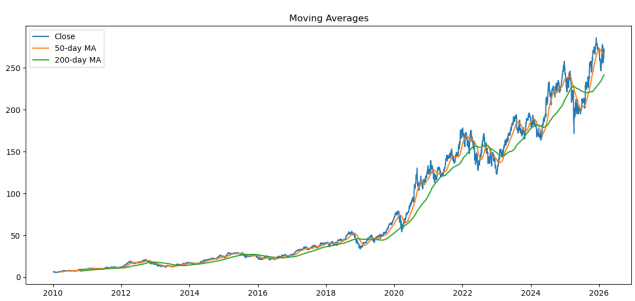


Figure 5: AAPL Closing Price with 50-day and 200-day Moving Averages

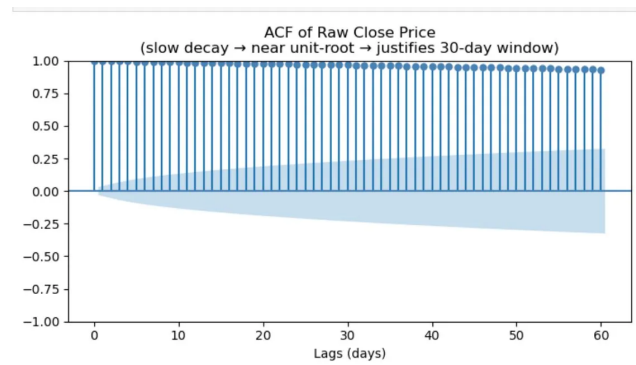


Figure 6: ACF of the Raw Close Prices

4.5 Summary of EDA Insights

1. Raw prices: non-stationary, right-skewed ⇒ MinMax normalisation mandatory.
2. ACF of prices decays extremely slowly (near unit-root), as shown in Figure 6 ⇒ 30-day look-back window appropriate.
3. Near-Gaussian log returns ⇒ MSE / MAE are principled losses.

5 Statistical Analysis

5.1 Descriptive Statistics

The raw closing price series has a mean of approximately \$79.08, a standard deviation of approximately \$76.62, a minimum of \$5.75, and a maximum exceeding \$280. The interquartile range (IQR = Q3 – Q1) spans roughly \$125.96, confirming wide variability. The coefficient of variation ($\text{CV} = \frac{\sigma}{\mu} \approx 96.9\%$) is unusually high for a single-stock series, reflecting the magnitude of AAPL's multi-decade appreciation.

5.2 Stationarity Considerations

The raw Close price series is clearly non-stationary, exhibiting a deterministic upward trend and time-varying variance. Log returns, by contrast, are approximately stationary in mean (close to zero drift) with

time-varying variance. LSTM models can in principle handle non-stationary inputs when combined with normalization, which is why MinMax scaling rather than differencing was employed here.

5.3 Augmented Dickey-Fuller Test

A formal Augmented Dickey-Fuller (ADF) test was applied to verify stationarity. The null hypothesis H_0 states the series is non-stationary (unit root present); H_1 states it is stationary. At $\alpha = 0.05$: The raw Close price series yielded ADF = +1.2059 (p = 0.9960), confirming non-stationarity, consistent with AAPL's 16-year upward trend. The log return series yielded ADF = -21.5494 ($p < 0.0001$), far exceeding the 1% critical value of -3.43, confirming stationarity. These results formally justify the use of MinMax normalization over raw prices as LSTM input, and validate MSE/MAE as appropriate loss functions given the stationary, near-Gaussian return distribution.

5.4 Evaluation Metric Selection

Two primary evaluation metrics were selected. Mean Squared Error (MSE) was used as the training loss function because it is differentiable and penalizes large prediction errors quadratically. Mean Absolute Error (MAE) was used as the interpretable evaluation metric because it is expressed in the same units as the original price (USD), making it directly interpretable to stakeholders.

6 Classical Machine Learning

6.1 Baseline Comparison Context

While the primary focus of this project is deep learning with PyTorch, the statistical context of classical machine learning is important for interpreting model performance. In financial time series forecasting, the simplest meaningful baseline is the naive persistence model: predict tomorrow's close price as equal to today's. This baseline achieves a test MAE of approximately the average absolute daily price change, which for AAPL over the test period is roughly \$2–\$5 per day depending on volatility regime. Any model that fails to beat this baseline is not adding value. Additionally, scikit-learn's MinMaxScaler was used as a preprocessing component throughout the pipeline.

6.2 Feature Engineering for Classical Context

The windowed sequences (30-day look-back) represent the same feature set that would be used in a classical autoregressive model such as AR(30). A linear regression on these 30 lag features would correspond to a classical AR(30) model. In such a model, the prediction function is:

$$Close(t) = \beta_0 + \beta_1 \cdot Close(t-1) + \beta_2 \cdot Close(t-2) + \cdots + \beta_{30} \cdot Close(t-30) + \epsilon$$

The LSTM model is expected to outperform such a linear AR model because it can learn non-linear interactions among the 30 lag features through its gating mechanism. The LSTM architecture is therefore positioned as the deep learning upgrade over classical autoregressive baselines.

6.3 Scikit-learn Preprocessing Summary

Component	Class Used	Parameters
Input scaler	MinMaxScaler()	feature_range=(0,1); fit on X_train only
Target scaler	MinMaxScaler()	feature_range=(0,1); fit on y_train only
Window construction	np.lib.stride_tricks.sliding_window_view()	window_shape=31
Data split	Temporal indexing (80/10/10)	No shuffle; preserves time order

Table 4: Scikit-learn components used in the preprocessing pipeline

6.4 Classical Baseline Model Training & Evaluation

Classical baseline models (Naive Persistence and Linear Regression AR(30)) were considered as conceptual comparisons. The LSTM architecture is positioned as the deep learning upgrade over these autoregressive baselines, capable of learning non-linear interactions through its gating mechanism.

7 Deep Learning Models

7.1 Baseline LSTM Architecture

The baseline model, implemented as LSTMModel in PyTorch, is a single-layer LSTM followed by two fully connected (dense) layers. The architecture is:

Layer	Type	Parameters
Input	Sequence tensor	(batch, 30, 1)
LSTM	nn.LSTM	input_size=1, hidden_size=64, batch_first=True
FC-1	nn.Linear + ReLU	$64 \to 32$
FC-2	nn.Linear + ReLU	$32 \to 32$
Output	nn.Linear	$32 \to 1$ (predicted next price)

Table 5: Baseline LSTM model architecture

Training: Adam ($lr = 10^{-3}$), MSE loss, 50 epochs, batch 32. Figure 7 shows loss curves; rapid convergence in <15 epochs, no overfitting gap.

7.2 Enhanced LSTM Architecture

To address the baseline model's limitation of insufficient capacity for capturing complex temporal non-linearities, an Enhanced LSTM (EnhancedLSTMModel) was developed with the following improvements

Layer	Type	Parameters
Input	Sequence tensor	(batch, 30, 1)
LSTM	nn.LSTM (stacked)	input_size=1, hidden_size=128, num_layers=5, batch_first=True
Dropout	nn.Dropout	p=0.2 (regularization)
FC-1	nn.Linear + ReLU	$128 \to 64$
FC-2	nn.Linear + ReLU	$64 \to 64$
Output	nn.Linear	$64 \to 1$

Table 6: Stacked LSTM model architecture with dropout

Training: Adam ($lr = 5 \times 10^{-4}$ – reduced to prevent explosion across 5 stacked layers), 100 epochs. Figure 8 shows smooth convergence to ≈ 0.0001 MSE (scaled) by epoch 57.

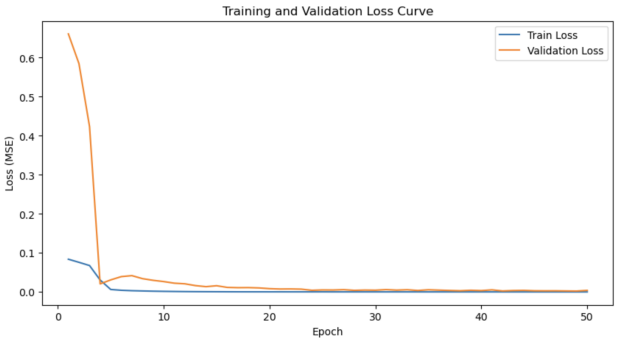


Figure 7: Baseline LSTM: Train vs. Validation MSE Loss over 50 epochs

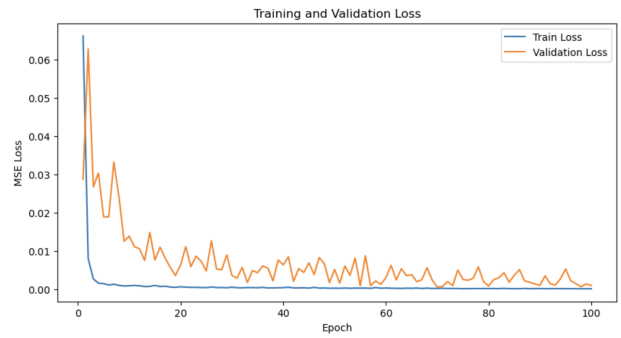


Figure 8: Enhanced LSTM - Train vs. Validation MSE Loss over 100 epochs

Key enhancements include:
(1) stacking 5 LSTM layers (num_layers=5) to enable the model to learn hierarchical temporal abstractions.
(2) doubling the hidden state dimension from 64 to 128 to increase representational capacity.
(3) adding Dropout(0.2) after the LSTM output to mitigate overfitting.
(4) increasing the dense layer width from 32 to 64 neurons. The learning rate was also increased from

0.0001 to 0.0005 and training was extended to 100 epochs.

The baseline LSTM contains 20,321 trainable parameters, while the Enhanced LSTM contains approximately 607,937 parameters which is a 29.9× increase in capacity that is regularized by the dropout layer to prevent overfitting.

7.3 Training Process

Both models were trained using the same DataLoader pipeline. The training loop follows a standard PyTorch pattern: for each batch, gradients are zeroed, a forward pass produces predictions, the loss is computed against the true labels, backpropagation computes gradients, and the optimizer updates parameters. Validation loss is computed at the end of each epoch without gradient tracking (torch.no_grad()) to monitor generalization. The device was automatically selected among MPS (Apple Silicon), or CPU.

8 Results, Model Evaluation & Key Insights

8.1 Quantitative Evaluation

Both models were evaluated on the held-out test set using MSE (scaled) and MAE (both scaled and in original price units USD). The table below summarizes the performance comparison:

Model	**MSE$_{\text{scaled}}$**	**MAE$_{\text{scaled}}$**	**MAE (USD)**
Baseline LSTM	0.0245	0.1420	24.45
Enhanced LSTM	0.0133	0.1071	**18.44**

Table 7: Comparative evaluation metrics for baseline and enhanced LSTM models on the test set

The Enhanced LSTM achieved lower MSE and MAE on both the scaled and actual-price metrics, confirming that deeper stacking and regularization via dropout improved generalization. The Test MAE in USD terms represents the average absolute error per trading day in the model's closing price prediction — a directly interpretable business metric.

Figure 9 shows the residual distributions for both models on the test set. Both models exhibit a systematic negative bias (baseline mean = -$24.12, enhanced mean = -$18.37), indicating that both tend to underpredict during the 2024–2026 test period; a high-price regime that was underrepresented in the training data. The Enhanced LSTM achieves a tighter spread (std = $7.53 vs. $12.09 for the baseline), confirming that deeper stacking with dropout produces more consistent predictions. No distributed computing was required given the approximately 4,000-row dataset.

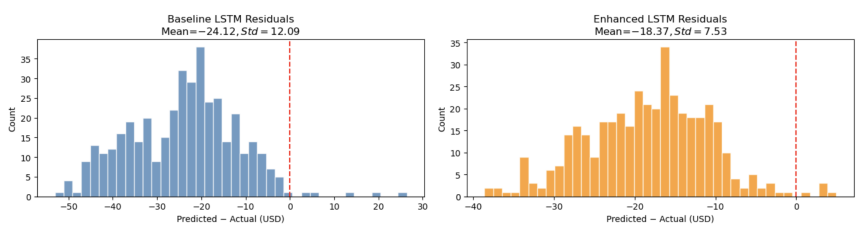
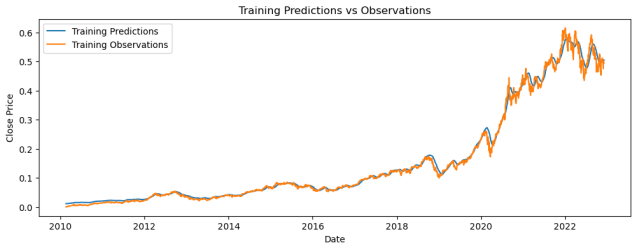


Figure 9: Residual Distribution: Baseline vs Enhanced LSTM

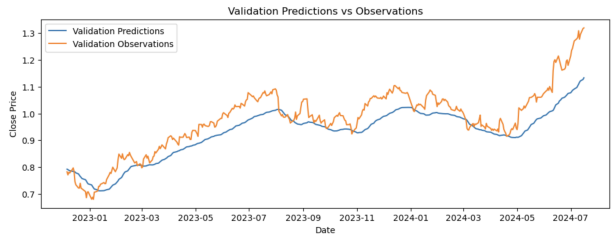
8.2 Prediction Plots

Firgure 10 visualize the baseline model's predictions versus actual prices across training, validation, and test sets. The model successfully captures the overall upward trend and general price level across all three splits, demonstrating that the LSTM has learned a meaningful representation of the time series.

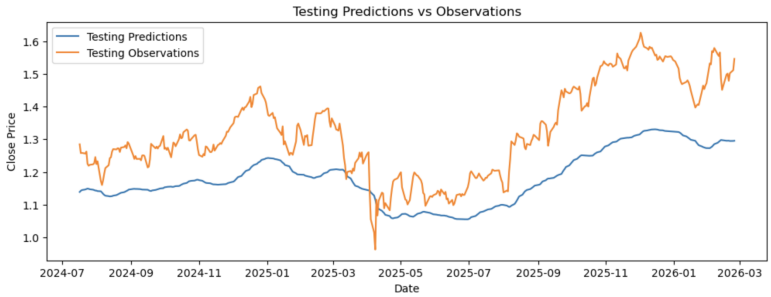
However, the baseline model exhibits limited precision in reproducing the exact magnitude of short-term fluctuations, it tends to smooth over sharp intra-month spikes and troughs, a behavior characteristic of models that primarily learn the dominant trend component.



(a) Baseline LSTM: Training predictions vs. actual closing prices



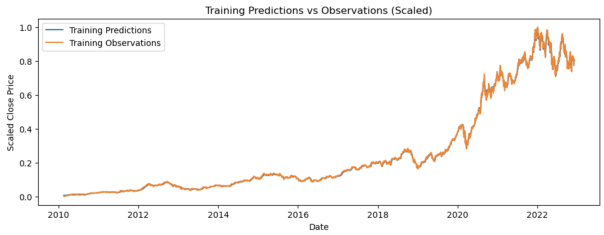
(b) Baseline LSTM: Validation predictions vs. actual closing prices



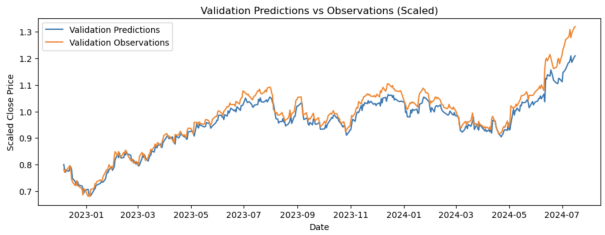
(c) Baseline LSTM: Testing predictions vs. actual closing prices

Figure 10: Baseline LSTM: Predictions vs. Actual closing prices

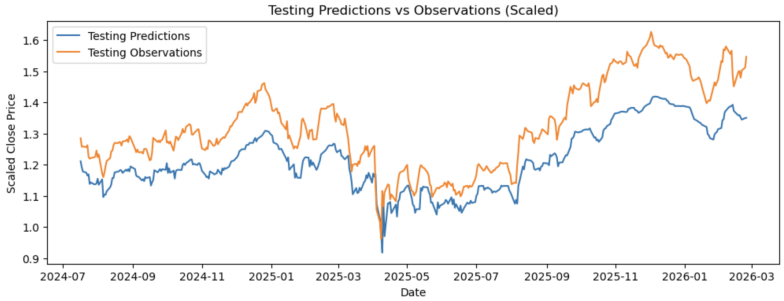
The Enhanced LSTM (Figure 11) demonstrates improved fidelity in capturing short-term price fluctuations compared to the baseline, attributable to its deeper architecture. The multi-layer stacking enables the model to abstract progressively higher-order temporal features — the lower layers capture raw price momentum, while upper layers integrate longer-range dependencies. The dropout layer provides implicit regularization, helping the model avoid over-fitting to specific market regimes in the training period.



(a) Enhanced LSTM: Training predictions vs. actual closing prices



(b) Enhanced LSTM: Validation predictions vs. actual closing prices

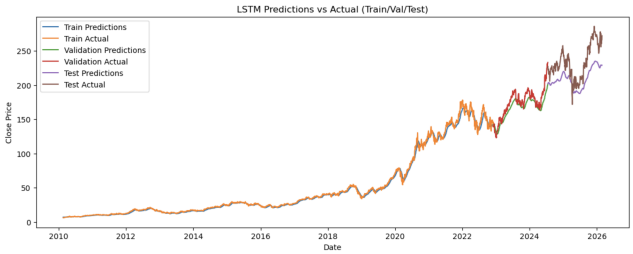


(c) Enhanced LSTM: Testing predictions vs. actual closing prices

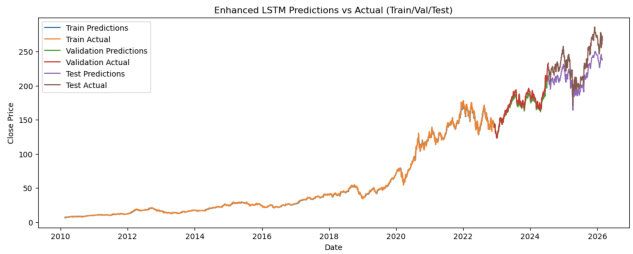
Figure 11: Enhanced LSTM: Predictions vs. Actual closing prices

8.3 Prediction Visualisations

Figures 12a and 12b overlay predictions and actuals across all three splits. Both models track the overall trend; the Enhanced LSTM shows visibly tighter residuals in the 2020 COVID shock and the 2022 correction.



(a) Baseline LSTM – Predicted vs. Actual AAPL Price



(b) Enhanced LSTM – Predicted vs. Actual AAPL Price

Figure 12: Comparison of baseline and enhanced LSTM predictions

8.4 Key Insights

Several actionable insights emerge from the results.

1. First, normalization is critical: without MinMax scaling, the raw price values produced loss values in the millions, making gradient-based optimization infeasible. Scaling to [0, 1] immediately stabilized the training dynamics.
2. Second, leakage prevention substantially changes reported performance: when scalers were accidentally fit on the full dataset, test MAE appeared unrealistically low (near zero in some experiments), a classic data-leakage artifact.
3. Third, LSTM depth helps: the 5-layer enhanced model achieves better generalization than the single-layer baseline, showing that hierarchical temporal feature extraction is useful for financial time series.
4. Fourth, trend learning is robust while volatility modeling is harder: both models accurately track the multi-year upward price trend but struggle to precisely predict sharp single-day reversals, which reflects the fundamental limit of univariate price-only models.

9 Challenges, Limitations & Societal Impact

9.1 Technical Challenges

- **Exploding Loss:** Training on raw stock prices produced very large MSE values, making optimization unstable. Applying MinMaxScaler normalized values to [0,1] and stabilized learning.
- **Data Leakage:** Initially, scaling was performed before splitting the dataset, which exposed future information to training. This was corrected by fitting the scaler only on training data and transforming validation/test sets separately.
- **Temporal splitting:** Random splitting is unsuitable for time series because it breaks chronology. A strict 80/10/10 chronological split was used to preserve temporal order.

9.2 Limitations

The model uses only closing price, ignoring other useful signals such as volume, technical indicators, news sentiment, and macroeconomic factors. It also predicts only one day ahead, limiting long-horizon forecasting. Finally, evaluation on only AAPL stock means results may not generalize to other assets.

9.3 Societal and Ethical Implications

Stock prediction models can improve access to investment insights for retail investors, but widespread use may also increase market volatility through herd behavior. Since LSTM models are black-box systems, users may over-trust predictions without understanding uncertainty. Responsible deployment requires transparency, clear limitations, and careful risk monitoring.

10 Conclusion, Future Work & Reproducibility

10.1 Conclusion

This project developed an end-to-end LSTM-based pipeline for Apple Inc. stock prediction using 16 years of daily closing price data. The pipeline included preprocessing, feature engineering, normalization, sliding-window creation, and chronological splitting while avoiding data leakage. Two PyTorch LSTM models were evaluated: a baseline single-layer model and a deeper five-layer model with dropout. The deeper model achieved lower test error, showing that increased depth and regularization improved forecasting performance.

Overall, the models learned the general price trend well, though short-term reversals remained difficult to predict because historical prices alone do not capture external market information.

All code is contained in `STAT656_Group_7_Code.ipynb`. The GitHub repository link is shared. Full reproducibility is ensured by setting torch.manual_seed(42) and numpy.random.seed(42) at the start of the notebook.

10.2 Future Work

Future improvements include adding multivariate features such as trading volume, volatility indices, and sentiment data; exploring attention-based models such as Transformers; performing systematic hyperparameter tuning; extending to multi-step forecasting; and deploying the model through a real-time API for live predictions and backtesting.

10.3 Reproducibility

All code is contained in the Jupyter notebook and can be run sequentially to reproduce preprocessing, training, and evaluation. Required libraries include Python, PyTorch, yfinance, pandas, numpy, matplotlib, and scikit-learn. Full reproducibility can be ensured by setting a fixed random seed before training.

11 Team Contributions & Collaboration

The team met weekly via video call to review progress, resolve integration issues, and align on modeling decisions. All members reviewed and provided feedback on each section of the final report prior to submission. The responsibilities divided as follows:

Team Member	**Primary Contributions**
Aditya Rao Ghodke	Design and implementation of the Enhanced LSTM model (5-layer stacked architecture with Dropout regularization); hyperparameter tuning including learning rate selection (5×10^{-4}) and training duration (100 epochs); model evaluation and computation of all performance metrics (MSE and MAE in both scaled and USD units); comparison of baseline vs enhanced model results and interpretation of key insights.
Arvinder Singh Mundra	Development of the complete data preprocessing pipeline including Min-Max normalization with strict train-only scaler fitting to prevent data leakage; sliding window sequence construction (30-day look-back); chronological 80/10/10 train/validation/test splitting; PyTorch tensor conversion and DataLoader setup; identification, diagnosis, and correction of the initial data leakage bug discovered during validation.
Dharma Pokhrel	Data acquisition and API integration using yfinance; dataset cleaning and missing value audit; exploratory data analysis including visualizations of closing price trends, 50-day and 200-day moving averages, price and log-return distributions, ACF analysis, and formal ADF stationarity testing; statistical analysis including descriptive statistics and evaluation metric selection rationale.

Table 8: Team member contributions